

Computer Hydrological Forecasting

Three independent monthly ARIMA models -- discharge, rainfall, stage -- validated
by stochastic properties, not point comparison

Author: Ugbodaga Benedict Osikpemi (190402003)

Department: Civil and Environmental Engineering

Institution: University of Lagos

Supervisor: Prof. K. O. Aiyesimoju

Submission: February 2026

This document explains the project in three parts: first in very simple terms, then the real technical version, and finally how to actually use the software. It starts with what changed most recently, since that's what makes everything after it correct.

0. What Changed Most Recently (and Why)

The modelling approach pivoted on 2026-08-12, on the supervisor's direct instruction, and it matters for reading everything that follows correctly.

The modelling approach pivoted

- **Monthly, not daily:** Differencing only does useful work -- removing seasonality -- at a monthly timestep, not a daily one, on a river that never really trends day to day.
- **Three variables, not one:** Discharge, rainfall and stage are each modelled independently, to show the same ARIMA method generalises across variable types, not just discharge.
- **Stochastic validation:** A stochastic model's individual forecasts are not meant to match the one thing that actually happened; only its statistical properties -- mean, variability, seasonality, drought duration, peak size -- should match history.

An even earlier version of this project (before either of the above) used a rainfall-runoff model on a Nigerian basin; that was replaced by the ARIMA approach on the same supervisor's instruction, and is not revisited here -- see CLAUDE.md and archive/ for that history if needed.

The report was then put through two rounds of critical review

After the pivot above, the thesis text was independently reviewed twice (each round: three reviewer personas instructed to challenge why, how, and where every claim came from, not just skim the document). Real, checkable problems came out of both rounds. A table-numbering gap, a data-vs-text arithmetic mismatch, and two backwards test-result claims (ARCH and Jarque-Bera) were fixed directly. Three deeper methodological gaps -- no worked reservoir/spillway design example, an unsupported cross-basin transfer claim, and an unstated discharge/stage independence caveat -- were resolved with real supplementary analysis rather than softened wording (see 'On generalisation and design use' in Section 2). Three other requests -- make the persistence skill score the headline metric, reintroduce an explicit Duan bias-correction step, and drop the qualification on the cross-basin claim -- were considered and declined, because each would have reintroduced exactly the point-forecast framing the 2026-08-12 pivot moved away from.

1. Questions and Answers From Last Review

This is the actual defence-style review session that drove the 2026-08-12 pivot above, extracted question by question from the recording, with what the project does about each one today. Some overlap with the curated questions list in the next section; this is the direct, literal record.

Q: Why are you using ARIMA? AR alone? MA alone? ARMA? ARIMA? ARIMAX? There are many variants -- why introduce all of them?

Each of those actually answers a slightly different question about how much a series remembers its own past. AR alone says only past values matter; MA alone says only past shocks matter; ARMA needs the series already stationary; ARIMA adds differencing when it isn't; ARIMAX would add an outside variable on top. I don't just pick ARIMA and stop there -- I compare all of these directly in my literature review, and then I let AIC choose the best structure for each variable from the data itself, not from my own preference.

-> *Where to look: Report Chapter 2, Section 2.4.*

Q: Why are you using daily data with ARIMA? You won't gain anything from the differencing... but if you difference monthly flows, you actually gain something, because it removes the seasonality.

That's fixed now -- I moved the whole pipeline to a monthly timestep. Daily discharge doesn't really have a seasonal cycle for differencing to remove; monthly does, so that's where the differencing actually does useful work.

-> *Where to look: Report Chapter 1, Section 1.5; Chapter 3, Section 3.1.*

Q: How do you estimate the parameters? The p, d, q you mentioned are not the parameters -- that's just the order. How do you estimate the autoregressive component? The moving average parameters? How did you get it?

You're right -- p, d, q are just the order, not the parameters. The actual parameters are the AR and MA coefficients, and I estimate those by conditional sum of squares. I don't just report a value for each one either -- every coefficient comes with a standard error, so it's clear how precisely I actually know it.

-> *Where to look: Report Chapter 3, Section 3.5; Chapter 4, Table 5; app 'For the curious' panel.*

Q: Can you even forecast? 'I can only forecast 7 days' -- that's terrible. Why?

That was a real bug in an earlier version -- a leftover default I've since removed. I can now forecast any future range I choose, with explicit 'from' and 'to' month-and-year controls.

-> *Where to look: App: Predict from / to controls.*

Q: Your data is up to 2014 -- how do you now know 2015 is correct? How do you check that? How do we know it's not garbage that you have forecasted?

I can't check one stochastic run against one real year and call it right or wrong -- every run is genuinely different by design. What I actually check is whether the real record's statistical

properties -- its mean, its variability, its seasonality, how long its dry spells run, how big its peaks are -- fall inside the range a thousand simulated versions produce.

-> *Where to look: Report Section 3.6, Section 4.6.*

Q: The data is bulky -- 35 years, 365 days a year -- is that bulky for a computer?

Not for a computer, no -- that was never really the issue. I moved to monthly not because of data volume, but because differencing only does useful work at a monthly step, which is what I just explained.

-> *Where to look: See the daily-vs-ARIMA answer above.*

Q: Which year to which year did you use -- 1980 to 2014?

Yes -- 420 months in total. I used 1980 to 2003 to identify the order and estimate the parameters, and I held back 2004 to 2014 completely, to validate the model on data it had never seen.

-> *Where to look: Report Chapter 3, Section 3.4.*

Q: When you're forecasting stochastic data, you can't compare to real data -- each forecast is different, so how can you compare it to 2015? It's meaningless. You can only compare the parameters -- the properties -- of what you forecast to the properties of the original data.

That's exactly right, and that's exactly how I validate it now. I never compare one synthetic run to the one thing that actually happened in 2015 -- I compare the statistical properties of the whole ensemble to the statistical properties of the historical record.

-> *Where to look: Report Section 4.6, Table 7.*

Q: Why are you using whatever river in the US? Have you checked for monthly data in Nigeria that you didn't find?

I did check -- I went directly to NIHSA, Nigeria's hydrological services agency. They couldn't give me a firm price or a firm delivery date inside my project deadline, so I built and validated the pipeline on the Conecuh record, which I already had in hand, verified, and clean. I kept the Nigerian request open the whole time -- I'm disclosing that trade-off, not hiding it.

-> *Where to look: Report Chapter 1, Section 1.5; 'Why Conecuh, specifically' below.*

Q: This business of 'not enough data' isn't really relevant -- it's precisely because you don't have enough data that you're forecasting. If you had huge data, why would you be forecasting at all?

I agree completely -- that's actually the whole justification for stochastic hydrology in my literature review. Thirty-five years isn't 'too little' or 'too much'; forecasting exists because the future is never in hand, no matter how much history you have.

-> *Where to look: Report Chapter 2 (Matalas, 1967).*

Q: Forecast rainfall from rainfall, runoff from runoff -- there's no difference. Your model doesn't change, it's the same model. You should be able to estimate the parameters

for whatever data you put in.

That's exactly how I built it. It's one ARIMA implementation, and I apply it completely unchanged to discharge, rainfall, and stage -- only the fitted coefficients differ between them, not the code.

-> *Where to look: Report Section 2.4, Section 3.2; app: switch tabs between the three variables.*

Q: You have to show that the ARIMA model fits the data -- ARIMA can't be used for just anything, you must show the data itself fits it, not assume it.

I do show that. Every variable goes through the ADF and KPSS stationarity tests before I choose a model, and I report the actual test evidence, not just my conclusion.

-> *Where to look: Report Section 3.5, Section 4.2; app 'For the curious' -> stationarity evidence line.*

2. Questions To Expect, And Where The Answer Lives

Every question below is one the supervisor has actually asked, in the exact defence session that drove this rewrite. Each answer is short on purpose -- say the sentence, then point at the app or the report page and let it do the rest of the talking.

On the choice of model

Q: Why ARIMA, and not AR alone, MA alone, ARMA, or ARIMAX?

AR alone assumes only past values matter; MA alone assumes only past random shocks matter; ARMA combines both but needs the series already stationary; ARIMA adds differencing on top for series that aren't; ARIMAX would add an outside variable. I don't pick by preference -- my order-selection step tests AR-only, MA-only and mixed forms for every one of the three variables, and keeps whichever the Akaike Information Criterion scores best. I deliberately ruled out ARIMAX: each variable is forecast from its own past only, so there's no outside variable for me to add.

-> *Where to look: Report Chapter 2, Section 2.4 (family comparison, explicit ARIMAX paragraph). App: each model's actual (p,d,q) is shown under 'For the curious'.*

Q: Shouldn't the same model work on rainfall too, not just discharge?

It does. The same code, completely unchanged, independently fits discharge, rainfall and river stage -- three different variables, three different fitted models, one shared method.

-> *Where to look: App: switch the River flow / Rainfall / River level tabs -- identical layout each time, different numbers. Report Chapter 2, Section 2.4; Chapter 3, Section 3.2.*

On daily versus monthly

Q: Why were you using daily data with ARIMA?

I'm not, any more. Differencing -- the 'I' in ARIMA -- only does real work when there's seasonal drift to remove, and that shows up at a monthly timestep, not day to day. I rebuilt the whole pipeline on monthly data for exactly that reason.

-> *Where to look: App: left-hand 'Good to know' panel. Report Chapter 1, Section 1.5; Chapter 3, Section 3.1.*

On parameter estimation (asked the most, and the most pointed)

Q: How do you estimate the parameters? p, d, q are not the parameters.

You're right, and I've corrected that. p, d, q are the order -- the shape of the model, chosen by AIC. The actual parameters are the phi (AR) and theta (MA) coefficients, estimated by conditional sum of squares -- exact least squares when there's no MA term. Every coefficient now carries a standard error too, so it's clear how precisely I actually know each one, not just its value.

-> *Where to look: App: 'For the curious' -> a coefficient table per variable, value and standard error side by side. Report Chapter 3, Section 3.5; Chapter 4, Section 4.4 (Table 5 and Figure 5).*

Q: You must show the data actually fits ARIMA -- not assume it.

I do. Every variable goes through the Augmented Dickey-Fuller and KPSS stationarity tests before I choose a model, and I show the actual result, not just my conclusion.

-> *Where to look: App: 'For the curious' -> stationarity evidence line per variable (ADF/KPSS statistics). Report Chapter 3, Section 3.5; Chapter 4, Section 4.2.*

On whether it can even forecast

Q: Can it forecast? Why is it limited to 7 days?

That cap is gone -- it was a leftover default, not a real limit of the method. The app now takes an explicit 'predict from [month/year] to [month/year]' range, not a horizon length off a fixed anchor, so I can target any future window -- 2030-2035, 2050-2060, whatever's asked for.

-> *Where to look: App: the 'Predict from / to' controls.*

On checking whether a forecast is correct

Q: Your data stops at 2014 -- how do you know a forecast into 2015 is correct? How do you check it?

I can't call a single stochastic forecast 'correct' or 'wrong' against one real sequence -- every run of the model produces a different plausible sequence, that's the nature of it. What I check instead is whether the real historical record's properties -- its average, its variability, its seasonal pattern, its drought length, its peak size -- fall inside the range a thousand simulated versions produce.

-> *Where to look: App: the shaded band on every chart, plus the 'track record' score on each card (e.g. 7/7). Report Chapter 3, Section 3.6; Chapter 4, Section 4.6.*

Q: When forecasting stochastic data, can't you only compare properties, not the forecast itself, to the original data?

Yes, exactly -- that's precisely what I changed. This isn't a point-forecast model scored against one observed sequence any more; it's a stochastic generator scored by whether its output's statistical properties match history.

-> *Where to look: Same as above -- see the previous answer.*

Q: Isn't 35 years of data 'not enough', or daily data 'too much' -- which is it?

Neither framing is really the point. Forecasting exists precisely because the future is never in hand, no matter how much history you have; 35 years is enough to estimate the model, and stochastic simulation is what you do with a finite past, not a reason to wait for more of it.

-> *Where to look: Report Chapter 2 (stochastic hydrology literature, Matalas 1967).*

On the study basin

Q: Why a US river? Why not a Nigerian one -- that's a standard question the panel will ask.

I did check the Nigerian option directly -- NIHSA specifically, see the next section for the actual

numbers. Short version: they couldn't give me a firm price or delivery date inside my project timeline, so I built and validated the pipeline on the Conecuh record, which I already had in hand, verified, and clean. I kept the Nigerian request open as a possible swap -- I'm disclosing that trade-off, not hiding it. And since then I've also tested whether my procedure generalises to other basins, with an honest, qualified answer -- I'll come to that.

-> *Where to look: Report Chapter 1, Section 1.5. See 'Why Conecuh, specifically' below for the full reasoning.*

Why Conecuh, specifically: the Nigerian data attempt

NIHSA (Nigeria Hydrological Services Agency) has a real online request form (nihsa.gov.ng/data-request) stating 5-7 working days turnaround for straightforward requests, but the request is not free -- cost is assessed per request and only communicated after NIHSA reviews it, so neither a firm price nor a true delivery date could be known in advance, inside a project deadline. CAMELS, by contrast, already provides free, ready-to-use monthly data for hundreds of US basins -- which is exactly what made it possible to test transferability on two more basins (the cross-basin check, Section 4.8) at zero cost and no wait. The method itself doesn't care which country supplied the numbers: it's the same ARIMA procedure applied to whatever series is in front of it, so if the Nigerian data comes through, it would run on a Nigerian basin exactly as it already runs on Conecuh and the two supplementary US basins. See DATA_OPTIONS.md for the full investigation.

On generalisation and design use (added after the 2026-08-13 review rounds)

Q: Does this actually transfer to other basins, or is it just this one?

I ran the exact same identification-and-estimation procedure, completely unmodified, on two more basins in climate regimes very different from Conecuh's -- one arid, in the interior West, and one humid continental with snow. It ran cleanly on both, for both discharge and rainfall, without touching the code. What didn't transfer was the specific pattern I found at Conecuh: discharge shows strong persistence at all three basins but fails its residual test at all three, and rainfall persistence genuinely differs by basin -- almost none at Conecuh, strong at the other two. Two of the four new fits also threw up real numerical warning signs, which I take as evidence my diagnostics are actually catching bad fits, rather than quietly reporting everything as fine.

-> *Where to look: Report Chapter 4, Section 4.8 (Table 9, Figure 9); [cross_basin_check.py](#), [cross_basin_figure.py](#).*

Q: You keep invoking reservoir and spillway design as the motivation -- can you actually show a design number?

Yes, I can. I generated five hundred synthetic thirty-year discharge traces from my fitted discharge model, pooled the annual maximum from every one of them -- fifteen thousand values in total -- and read return-period design discharges straight off that pooled distribution, using the standard plotting-position method (Chow, Maidment and Mays, 2008). My hundred-year design discharge comes out to about 376 cubic metres per second, against an observed 35-year peak of

about 110. I want to be upfront that this is illustrative -- a real design study would use a formally chosen exceedance-probability standard and cross-check several methods -- but it is a real, reproducible calculation, not just an invoked application.

-> *Where to look: Report Chapter 4, Section 4.6 (Table 8); [design_discharge_example.py](#).*

3. The Simple Version (for anyone)

A river does not change from one month to the next by pure chance. A wet month tends to follow other wet months in the same season; a dry spell tends to persist for a while once it starts. This project builds three small computer models -- one for how much water flows in the river, one for how much rain falls on its catchment, and one for how high the river runs -- and each one learns its own variable's habits from decades of monthly history. None of them needs a weather forecast; each works from its own past alone.

Because rivers, rainfall and river levels are naturally unpredictable (that's what "stochastic" means here), the model doesn't try to name the one exact number that will happen next month. Instead it generates a thousand plausible versions of what could happen, and the honest question asked of it is: do the ordinary and extreme features of those thousand versions -- how wet, how dry, how variable, how seasonal -- look like the real history? That is a much fairer test of a model that admits the future is uncertain than asking it to guess one specific number correctly.

How do we know it works?

For each of the three variables, we held back eleven years of real data (2004-2014) that the model never saw while being built, generated a thousand synthetic versions of what those eleven years could have looked like, and checked seven different properties of the real data -- its average, its variability, how extreme its wettest and driest spells were, and so on -- against the range the thousand synthetic versions produced. Discharge and rainfall passed on all seven; river stage passed on six of seven, and the one it missed (its average level) is reported honestly rather than hidden.

4. The Actual Version (technical)

The project is a modular, open-source Python framework for monthly hydrological forecasting. It fits three independent ARIMA (autoregressive integrated moving average) models, one each for discharge, rainfall and stage, using the Box-Jenkins methodology, and validates each by the statistical properties its stochastic output reproduces relative to the historical record. No variable is used to forecast another, and no exogenous (meteorological forecast) input is used.

4.1 The data

Monthly discharge (m³/s), rainfall (mm/month) and stage (m) for the Conecuh River at Brantley, Alabama (USGS gauge 02371500 -- verified directly against USGS NWIS, see page 2), 1980-2014 (420 months). Discharge and stage come from USGS NWIS; rainfall is the Daymet basin-mean product from the CAMELS archive, chosen because it is the only one of three available rainfall products with zero missing days across the full record. All three series are strictly positive throughout, so each is modelled on the natural-log scale. The record is split into a training period (1980-2003) and an independent validation period (2004-2014).

4.2 The model: three independent ARIMA(p, d, q)

An ARIMA model describes a series using three ingredients: an autoregressive part (the value depends on its own p past values), an integration order d (the series is differenced d times to make it stationary), and a moving-average part (the value depends on the q past random shocks). On the differenced log-series $w(t)$, for each variable independently:

$$w(t) = c + \phi_1 w(t-1) + \dots + \phi_p w(t-p) + a(t) + \theta_1 a(t-1) + \dots + \theta_q a(t-q)$$

where ϕ are the autoregressive coefficients, θ the moving-average coefficients, c a constant, and $a(t)$ a white-noise error term. ARIMAX (which adds an exogenous predictor) was deliberately not used: each variable is forecast from its own past only, by design, so there is no exogenous driver to add.

4.3 How each model was identified (Box-Jenkins)

- **Stationarity:** Augmented Dickey-Fuller and KPSS tests determined the differencing order for each variable independently. For this basin, at the monthly timestep, all three came back $d = 0$.
- **Identification:** The autocorrelation (ACF) and partial autocorrelation (PACF) functions suggested candidate AR and MA orders.
- **Estimation:** Coefficients were estimated by conditional sum of squares (pure AR models by exact least squares), each with a standard error -- answering not just what order was picked, but how precisely each coefficient is known.
- **Selection:** All candidate orders were ranked by the Akaike Information Criterion (AIC).

Discharge : ARIMA(4, 0, 1) AIC=-139.3

Rainfall	: ARIMA(1, 0, 0)	AIC=-258.2
Stage	: ARIMA(4, 0, 0)	AIC=-472.9

4.4 Stochastic ensembles and property-based validation

Rather than one deterministic forecast, each fitted model generates an ensemble of 1,000 synthetic monthly sequences over the validation period. Each sequence, and the actual historical record, is characterised by seven statistics: mean, standard deviation, skewness, month-to-month persistence, seasonal amplitude, longest dry spell, and peak value. A statistic is judged reproduced if the historical value falls within the ensemble's 5th-to-95th-percentile range.

Variable	Properties within 90% envelope
Discharge	7 / 7
Rainfall	7 / 7
Stage	6 / 7

Stage's one miss (its mean) is traced to residual seasonal structure the non-seasonal ARIMA specification does not fully capture -- visible independently as a failed Ljung-Box residual test for discharge and stage ($p < 0.05$, some autocorrelation remains) but not for rainfall ($p = 0.960$). This is reported as an acknowledged limitation, not concealed: a validation procedure that always passes would not be doing useful work.

4.5 Code structure

```
src/preprocess.py - monthly loaders: discharge, rainfall, stage
src/model.py      - ARIMA + ADF/KPSS/ACF/PACF/Ljung-Box + std. errors
src/calibrate.py  - stationarity tests + AIC order selection
src/simulate.py   - stochastic synthetic-ensemble generation
src/validation.py - property-based validation vs. historical record
src/plots.py      - the six figures (3-variable, monthly, stochastic)
run_pipeline.py   - runs everything end to end, all 3 variables
write_full_report.py + report_*.py - builds the full Ch1-5 report
app.py + pages/   - the Streamlit web app
```

The entire time-series toolkit -- ARIMA, stationarity tests, ACF/PACF, Ljung-Box, standard errors, stochastic simulation, property-based validation -- is implemented directly from first principles on NumPy and SciPy, so the framework is fully self-contained and reproducible (no external time-series library is required).

5. How to Use the Software

5.0 The easy version

Open the app -- it's called "River Outlook" in the top navigation. Pick a future month-and-year range to predict, and press "Get the Outlook". One click forecasts all three variables together -- you don't pick one first. You'll see bands of plausible futures, not single lines -- and they'll look slightly different every time you press the button, because the model is honestly stochastic. Here is all you do:

- **1.** Open the app - a page opens in your web browser.
- **2.** In the centre panel, pick a range preset or set "Predict from" and "to" month and year directly.
- **3.** Press the "Get the Outlook" button.
- **4.** Read the answer: a compact outlook card for each of the three variables on the right, and a chart with the plain-language story behind each one in the centre.

5.1 The web app, step by step (the main way to use it)

Open the app and you get a browser page with two tabs along the top: "River Outlook" and "How This Works". The River Outlook page is laid out in three panels -- a left panel with context and quick facts, a centre panel with the controls, charts and plain-language story, and a right panel with a compact outlook card per variable. On the River Outlook page:

- **1.** Pick a jump-to-range preset (e.g. "2030 - 2035"), or set "Predict from" and "to" directly with the month dropdown and year field (year has +/- stepper buttons, from 2015 up to 2200).
- **2.** Click "Get the Outlook" -- this runs all three variables (discharge, rainfall, stage) at once, not one at a time.
- **3.** Read the results: a compact card per variable on the right (trend, level, track record), and in the centre a tab per variable with a chart (recent observed history up to Dec 2014, the synthetic uncertainty band, and the expected path) plus a fill-in-the-blanks explanation sentence underneath it.
- **4.** Open "For the curious" at the bottom of the centre panel for the actual numbers behind each model: the AR/MA coefficients with their standard errors, the stationarity test evidence, and the full property-based validation table.

5.2 The full run (get every chart and number at once)

Run the whole study in one command. It loads all three monthly series, runs the stationarity tests, identifies and estimates all three ARIMA models with standard errors, generates the stochastic ensembles, runs property-based validation, and saves all six figures plus a results file (data/results.json).

```
python run_pipeline.py      # all 3 models + ensembles + figures
python write_full_report.py # rebuild the full Ch1-5 report .docx
python make_overview_pdf.py # rebuild this overview PDF
streamlit run app.py        # launch the interactive web app
```